

I užduotis (Dirbtinis neuronas)

Pijus Petkevičius

2024 m. rugsėjo 15 d.

Turinys

1 Įvadas	3
1.1 Tikslas	3
1.2 Uždaviniai	3
2 Klasifikavimo duomenys ir klasė	3
3 Programos kodas	4
4 Svorį ir poslinkių parinkimas	6
4.1 Parinkimo strategija	6
4.2 Rezultatai	6
4.2.1 Rezultatai su slenkstine aktyvacijos funkcija	6
4.2.2 Rezultatai su sigmoidine aktyvacijos funkcija	6
4.3 Nelygybių sistema	7
4.4 Nelygybių sistemos sprendinio patikrinimas	8
5 Išvados	9

1 Įvadas

1.1 Tikslas

Užduoties tikslas- išanalizuoti dirbtinio neurono modelį, jo veikimo principus, susipažinti su dirbtinių neuronų pagrindais.

1.2 Uždaviniai

- Parašyti pasirinkta programavimo kalba dirbtinio neurono realizaciją. Programa turi rasti poslinkio ir svorių reikšmes, naudojant slenkstinę ir sigmoidinę funkcijas, kad 1 lentelėje esantys duomenys būtų tinkamai klasifikuoti.
- Užrašyti nelygybių sistemą, kurią reikia spręsti, norint parinkti tinkamas poslinki ir svorių reikšmes, kai aktyvacijos funkcija yra slenkstinė.
- Patikrinti grafiniu būdu nubraižytą nelygybių sistemą, ar sprendiniai, gauti iš grafiko, tenkina nelygybių sistemą.

2 Klasifikavimo duomenys ir klasė

Šiame darbe 1 lentelėje atvaizduotiems duomenims ieškomas poslinkis ir svoriai, su kuriais būtų teisingai klasifikuojami.

1 lentelė: Duomenys klasifikavimui

Duomenys		Klasė
x_1	x_2	t
-0,2	0,5	0
0,2	-0,7	0
0,8	-0,8	1
0,8	1	1

3 Programos kodas

```
1 import numpy as np
2
3 # Paimamos įvesties ir svorių reikšmės, sudauginama i-oji
4   ↳ įvesties reikšmė su i-ojo svorio reikšme ir taip su visomis
5   ↳ įvesties reikšmėmis, jos susumuojamos ir grąžinama reikšmė
6 # (arba galima panaudoti scalarinių sandaugą)
7 def countA(inputValues: [float], weights: [float] ) -> float:
8     return np.dot(inputValues, weights)
9
10 # Apskaičiuojama slenkstinės funkcijos reikšmė
11 def threshold(x) -> int:
12     return 1 if x >= 0 else 0
13
14 # Apskaičiuojama sigmoidinės funkcijos reikšmė
15 def sigmoid(x) :
16     return round(1 / (1 + np.exp(-x)))
17
18 # Paimamas įvesties duomenų masyvas, atskiriama įvesties reikšmė
19   ↳ nuo išvesties reikšmės. Prie įvesties pridedama 1(poslinkio
20   ↳ reikšmę norint traktuoti kaip svorį).
21 def separateInputOutputAndAddBias(inputValues: np.ndarray) ->
22   ↳ (np.ndarray, [float]):
23     output = inputValues[:, -1]
24     removedOutput = np.delete(inputValues, -1, axis=1)
25     inputValues = np.concatenate((np.array([1 for _ in
26       ↳ range(removedOutput.shape[0])])[:, np.newaxis],
27       ↳ removedOutput), axis=1)
28     return inputValues, output
29
30 # Šioje funkcijoje kiekvienai įvesties reikšmei pritaikoma countA
31   ↳ funkcija kartu su aktyvacijos funkcija ir tikrinama ar gauta
32   ↳ reikšmė sutampa su išvesties reikšme.
33 def applyWeightstoAllInputs(inputValues: np.ndarray, output:
34   ↳ [float], weights:[float], activationFunction: callable) ->
35   ↳ bool:
36     countMatches = 0
37     for i in range(inputValues.shape[0]):
38         if activationFunction(countA(inputValues[i], weights)) ==
39           ↳ output[i]:
40             countMatches += 1
41
42     return countMatches == inputValues.shape[0]
43
44 def readDataFromFile(filename: str) -> np.ndarray:
```

```

33     return np.loadtxt(filename, delimiter=',', skiprows=1)
34
35     # Sugeneruojami visi galimi svorių variantai intervale [-10, 10]
36     ↪ su žingsniu 1
37     def generateWeights() -> [float, float, float]:
38         values = np.arange(-10, 10, 1)
39         weights = []
40         for i in values:
41             for j in values:
42                 for k in values:
43                     weights.append([i, j, k])
44         return weights
45
46     # Funkcija paima visus sugeneruotus svorių kombinacijas, pritaiko
47     ↪ investies reikšmėms, patikrina ar applyWeightstoAllInputs
48     ↪ grąžina True, jei taip, tai prideda svorių kombinaciją prie
49     ↪ tinkamų svorių masyvo
50     def weightSearchFunction(input: np.ndarray, output: float,
51                               weights: [float, float, float], function: callable) ->
52     [float, float, float]:
53         validWeights = []
54         for weight in weights:
55             if applyWeightstoAllInputs(input, output, weight,
56                                         function):
57                 validWeights.append(weight)
58         return validWeights
59
60     inputData = readDataFromFile('input.csv')
61     input, output = separateInputOutputAndAddBias(inputData)
62     generateWeights = generateWeights()
63
64     print("Threshold")
65     print(weightSearchFunction(inputData, output, generateWeights,
66                               ↪ threshold))
67
68     print("Sigmoid")
69     print(weightSearchFunction(inputData, output, generateWeights,
70                               ↪ sigmoid))

```

4 Svių ir poslinkių parinkimas

4.1 Parinkimo strategija

Su python programavimo kalba sukurta **generateWeights()** funkcija, kuri sugeneruoja visas intervale $[-10,10)$ reikšmes(eil. 36-43). Vėliau su **weightSearchFunction(...)** funkcija kiekviena svių kombinacija yra pritaikoma įvesties duomenims, patikrinama ar pritaikius aktyvacijos funkciją(slenkstinė [eil. 9-10](#) arba sigmoidinė [eil. 13-14](#), visos gautos reikšmės sutampa su tikėtina klasės reikšme ([eil. 46-51](#)).

4.2 Rezultatai

Su slenkstine aktyvacijos funkcija gauti 52, o su sigmoidine 72 svių ir poslinkių rinkiniai(Jei sigmoidinė funkcija nebūtų apvalinama, būtų gaunama 0 svių ir poslinkio reikšmių kombinacijų),

4.2.1 Rezultatai su slenkstine aktyvacijos funkcija

[2 lentelėje](#) pateiktos svių ir poslinkio reikšmės, su kuriomis slenkstinė aktyvacijos funkcija tinkamai suklasifikavo duomenis.

2 lentelė: Poslinkio ir svių rinkiniai su slenkstine aktyvacijos funkcija

w_0	w_1	w_2
3	1	-1
3	1	0
3	1	1
3	1	2
6	2	-1

4.2.2 Rezultatai su sigmoidine aktyvacijos funkcija

[3 lentelėje](#) pateiktos svių ir poslinkio reikšmės, su kuriomis sigmoidinė aktyvacijos funkcija tinkamai suklasifikavo duomenis.

Šioje užduotyje sigmoidinės funkcijos reikšmė yra apvalinama iki sveikojo skaičiaus vienetų tikslumu(0 arba 1). Dėl šios priežasties naudojant sigmoidinės aktyvacijos funkciją, buvo gauta daugiau svių ir poslinkio reikšmių; didesnė paklaida.

3 lentelė: Poslinkio ir svorių rinkiniai su sigmoidine aktyvacijos funkcija

w_0	w_1	w_2
5	2	2
5	2	3
5	2	4
5	2	5
6	2	4

4.3 Nelygybių sistema

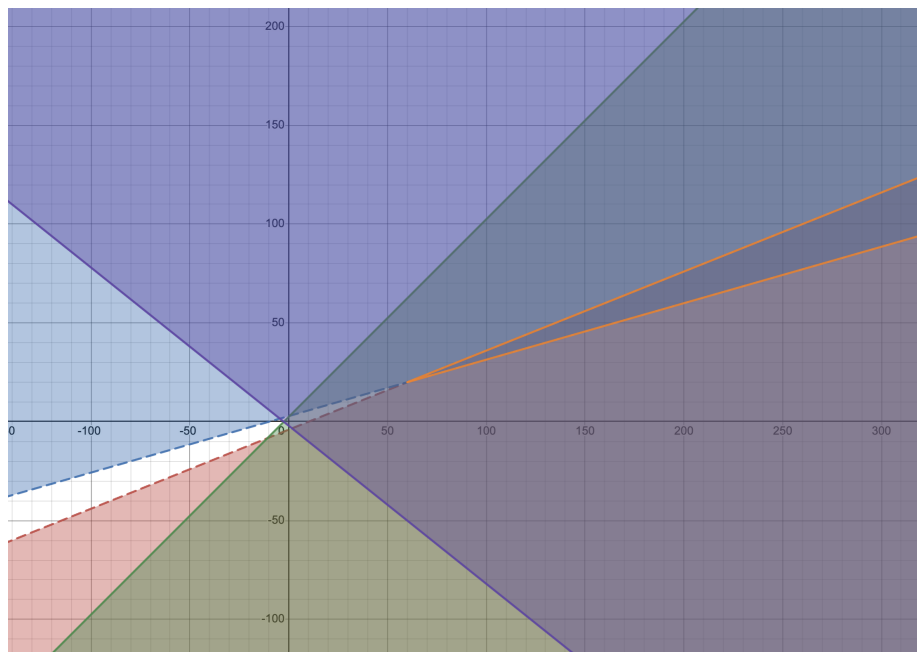
Norint teisingai parinkti svorių ir poslinkio reikšmes, kai aktyvacijos funkcija yra slenkstinė, reikėtų išspręsti tokią nelygybių sistemą:

$$\begin{aligned}
 w_0 - 0,2w_1 + 0,5w_2 &< 0, \\
 w_0 + 0,2w_1 - 0,7w_2 &< 0, \\
 w_0 + 0,8w_1 - 0,8w_2 &\geq 0, \\
 w_0 + 0,8w_1 + 1w_2 &\geq 0
 \end{aligned}$$

Kadangi nelygybių sistema sprendžiama grafiškai(nenorime naudoti 3d erdvės), parinkime w_0 reikšmę $w_0 = 2$ Gauta tokia nelygybių sistema:

$$\begin{aligned}
 2 - 0,2w_1 + 0,5w_2 &< 0, \\
 2 + 0,2w_1 - 0,7w_2 &< 0, \\
 2 + 0,8w_1 - 0,8w_2 &\geq 0, \\
 2 + 0,8w_1 + 1w_2 &\geq 0
 \end{aligned}$$

Šios nelygybių sistemos sprendinys matomas 1 pav. pažymėta oranžine spalva.



1 pav.: Nelygybių sistemos sprendinys

4.4 Nelygybių sistemos sprendinio patikrinimas

. Iš 1 pav. parinkti 2 sprendiniai, tenkinantys nelygybių sistemą:

Kai $w_1 = 150$ ir $w_2 = 50$:

$$2 - 0,2 \cdot 150 + 0,5 \cdot 50 = 2 - 30 + 25 = -3 < 0,$$

$$2 + 0,2 \cdot 150 - 0,7 \cdot 50 = 2 + 30 - 35 = -3 < 0,$$

$$2 + 0,8 \cdot 150 - 0,8 \cdot 50 = 2 + 120 - 40 = 82 \geq 0,$$

$$2 + 0,8 \cdot 150 + 1 \cdot 50 = 2 + 120 + 50 = 172 \geq 0$$

Kai $w_1 = 250$ ir $w_2 = 75$:

$$2 - 0,2 \cdot 250 + 0,5 \cdot 75 = 2 - 50 + 37,5 = -10,5 < 0,$$

$$2 + 0,2 \cdot 250 - 0,7 \cdot 75 = 2 + 50 - 52,5 = 0,5 < 0,$$

$$2 + 0,8 \cdot 250 - 0,8 \cdot 75 = 2 + 200 - 60 = 142 \geq 0,$$

$$2 + 0,8 \cdot 250 + 1 \cdot 75 = 2 + 200 + 75 = 277 \geq 0$$

5 Išvados

- Naudojant sigmoidinę aktyvacijos funkciją, galime gauti reikšmes intervale $(0,1)$, kas tam tikruose uždaviniuose būtų žymiai tiksliau nei slenkstinė aktyvacijos funkcija.
- Nors ir parinkus intervalą $[-10,10)$ su žingsniu 1, slenkstinės ar sigmoidinės aktyvacijos funkcijos pagalba gaunama 52 ir 72 reikšmės. Didinant intervalą ar mažinant žingsnio reikšmę, galime tikėtis didesnio svorių ir poslinkių kiekio.